

- The goal is to make security capabilities available at the point of development.
- The source uses this as a reference model for a possible AppScan integration.
- The integration is broader than MCP alone because workflow mechanisms are added around the MCP capability layer.

Important

The document is not saying MCP itself is a security workflow engine. MCP provides the standardized connection to the capability.

2. MCP: Client, Server and Protocol

MCP stands for Model Context Protocol. The document describes it as the standardized mechanism that lets an AI application communicate with external tools/services.

Component	Role
Claude Code	MCP client and AI coding agent.
MCP	Standard communication/interface layer.
Snyk MCP Server	MCP server exposing Snyk capabilities.
Snyk	Provides the security functionality behind those tools.

What MCP is not

- Not a scanner.
- Not a vulnerability database.
- Not an AI model.
- Not the security engine itself.

Easy analogy

Think of MCP as a standardized socket. It lets Claude plug into an external capability; it does not generate the capability itself.

3. How Claude Starts Snyk MCP

The source shows Snyk configured as an MCP server that Claude Code can launch locally using STDIO.

```
{
  "mcpServers": {
    "Snyk": {
      "command": "npx",
      "args": ["-y", "snyk@latest", "mcp", "-t", "stdio"]
    }
  }
}
```

Part	Meaning
npx	Runs an npm package.
snyk@latest	Snyk CLI package.
mcp	Starts Snyk MCP functionality.
-t stdio	Uses STDIO transport in the shown setup.
Claude Code Launches local Snyk MCP process	
↓	
Snyk MCP Runs as a local process	
↓	
STDIO Client/server communicate over standard input/output	
Current-document wording The original 51-page document contains an absolute statement that SSE is retired/rejected. For a current-facing summary, that statement should not be treated as a universal fact; this summary focuses on the STDIO setup shown.	

4. What Tools Does Snyk Expose?

The source emphasizes specialized, semantically meaningful tools instead of one generic security function.

Tool	Purpose in the analysis
snyk_auth	Authentication
snyk_logout	Logout
snyk_version	Version information
snyk_trust	Local folder trust
snyk_code_scan	SAST / code security
snyk_sca_scan	Open-source dependency security
snyk_iac_scan	IaC security
snyk_container_scan	Container security
snyk_sbom_scan	SBOM analysis
snyk_send_feedback	Feedback

Why specialized tools?

- Java code changed → code scan is relevant.
- Maven dependency added → SCA is relevant.
- Terraform changed → IaC scan is relevant.

AppScan lesson

Expose focused capabilities such as code scan, DAST scan, findings retrieval, authentication and project information rather than one oversized generic tool.

5. Example: Scan a Java Project

The source uses a project containing Java source files and a Maven `pom.xml`. The developer asks: “Scan this project for security vulnerabilities.”

```
my-project/  
  src/  
    LoginController.java  
    UserService.java  
  pom.xml
```



Conceptual SAST request from the source:

```
{  
  "tool": "snyk_code_scan",  
  "arguments": { "path": "/Users/developer/my-project" }  
}
```

Key idea

The developer expresses intent naturally. Claude has meaningful tools available and can select the relevant security capability.

6. Inside the MCP Server: End-to-End Flow

The document describes the Snyk MCP server as a controlled bridge between Claude’s structured tool request and the Snyk CLI.



7. Authentication and Folder Trust

Two controls highlighted by the document are authentication and local folder trust.

Authentication

- `snyk\_auth` is exposed as an explicit MCP capability.
- The source also describes optional `SNYK\_TOKEN` configuration/fallback.
- The integration handles authentication as part of the tool experience.

Folder trust

- `snyk\_trust` controls which local folders Snyk is permitted to scan.
- The MCP implementation checks trust before normal scan execution.
- An untrusted folder can result in instructions to trust it first.



8. Tool Descriptions Are Part of the AI Design

The analysis stresses that an MCP tool is not just a function name. Its description/schema gives the LLM the information needed to choose and invoke it correctly.

Description element	Example
Purpose	Performs SAST on application source code.
When to use	Local development / after generated code.
Required	Absolute path.
Optional	Organization, severity threshold, ignored issues.
Constraints	Parameter rules and expected behavior.

A weak tool description

```
snyk_code_scan
Performs SAST on source code.
```

A useful description also tells Claude when to use it, what inputs are required, what is optional and what constraints apply.

AppScan lesson

Tool schemas are part of the AI integration design. They should be deliberately written, not treated as API documentation pasted in at the end.

9. Why Scan Results Are Adapted

The source says the MCP server processes/enhances scan output before returning it to Claude.

```
graph TD; A[Raw CLI output] --> B[enhanceOutput()]; B --> C[mapScanResponse()]; C --> D[MCP result];
```

The diagram illustrates a four-stage process for handling scan results. It begins with 'Raw CLI output', which is noted as being 'Potentially large / implementation-specific'. This stage leads to 'enhanceOutput()', an 'Enhancement stage'. The next stage is 'mapScanResponse()', which provides 'Map scan information'. Finally, the process concludes with the 'MCP result', which is described as being 'More useful to the AI agent'. Each stage is connected to the next by a downward-pointing arrow.

- The exact response structure can vary by tool and implementation.
- The important architectural point is the adapter role.
- Avoid overwhelming the model with unnecessary raw output.

AppScan lesson

Normalize AppScan findings into concise, structured information with enough context for Claude to reason about remediation.

10. MCP vs Secure-at-Inception

This distinction is the most important conceptual lesson in the document.

Layer	Question answered	Examples
MCP capability	What can Claude do?	code scan, SCA, IaC scan
Rules / directives	When/how should it be used?	Security guidance
Hooks / plugin	What happens around agent events?	Background scans, enforcement, remediation
MCP Makes capabilities available		
↓		
Directives / rules Guide intended usage		
↓		
Hooks / plugin Connect checks to agent events		
↓		
Secure-at-Inception Turns capability into workflow		
Critical distinction MCP alone does not mean “run a scan after every code modification.” That behavior comes from the surrounding workflow mechanisms.		

11. Secure-at-Inception: Scan → Fix → Re-scan

The source describes the current Snyk Claude plugin as adding hooks and workflow components around the MCP capabilities.

1. File modification

Claude changes a workspace file



2. Background scan

Snyk checks the change



3. Evaluate

Look for newly introduced vulnerabilities



4. If found

Provide findings to Claude



5. Remediate

Claude changes the code



6. Re-scan

Validate the change



7. Continue

Implementation caps the remediation cycle at three rounds

- No findings → Claude can finish normally.
- Findings → Claude is prevented from simply stopping and is given security feedback.
- Dependency-manifest changes can also trigger SCA in the documented plugin workflow.

Why it matters

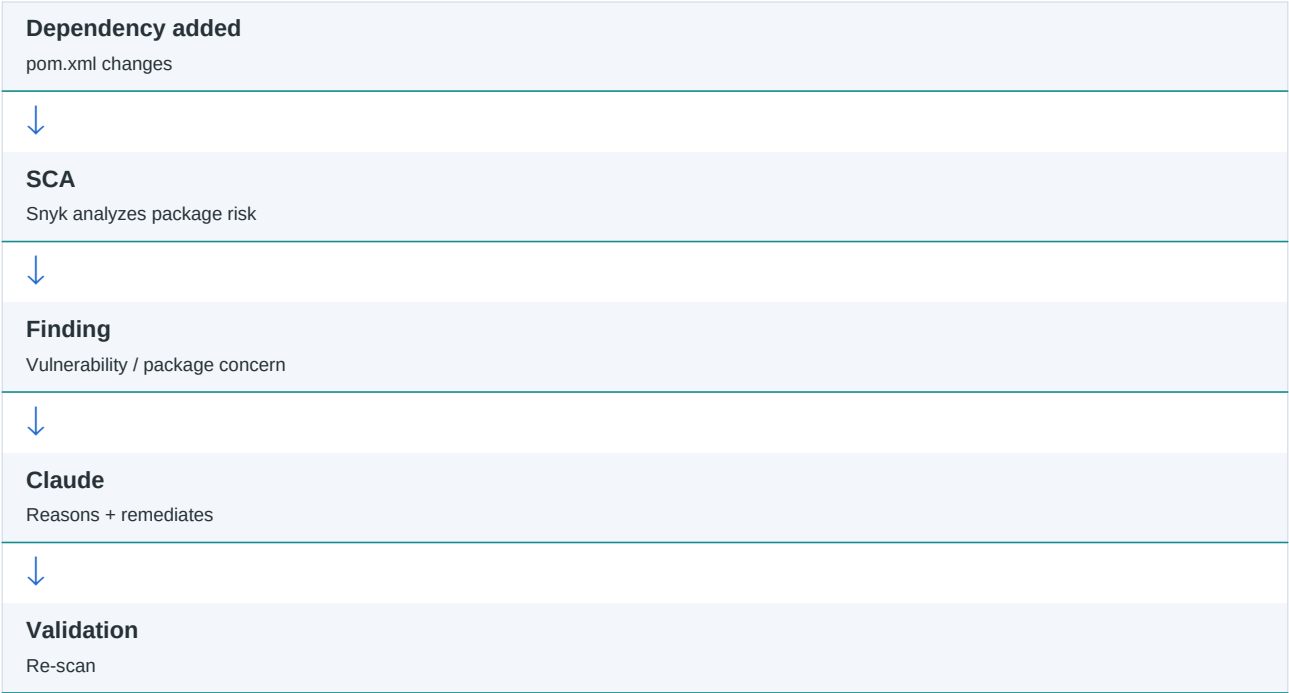
Security becomes part of the agentic development loop rather than a separate post-development scan.

12. Two Concrete Examples

Example A — vulnerable dependency

Developer: “Add the latest version of package X.” Claude modifies `pom.xml`. The plugin can detect the dependency-manifest change and trigger SCA.

```
<dependency>
  <groupId>example</groupId>
  <artifactId>package-x</artifactId>
  <version>1.2.0</version>
</dependency>
```



Example B — vulnerable code

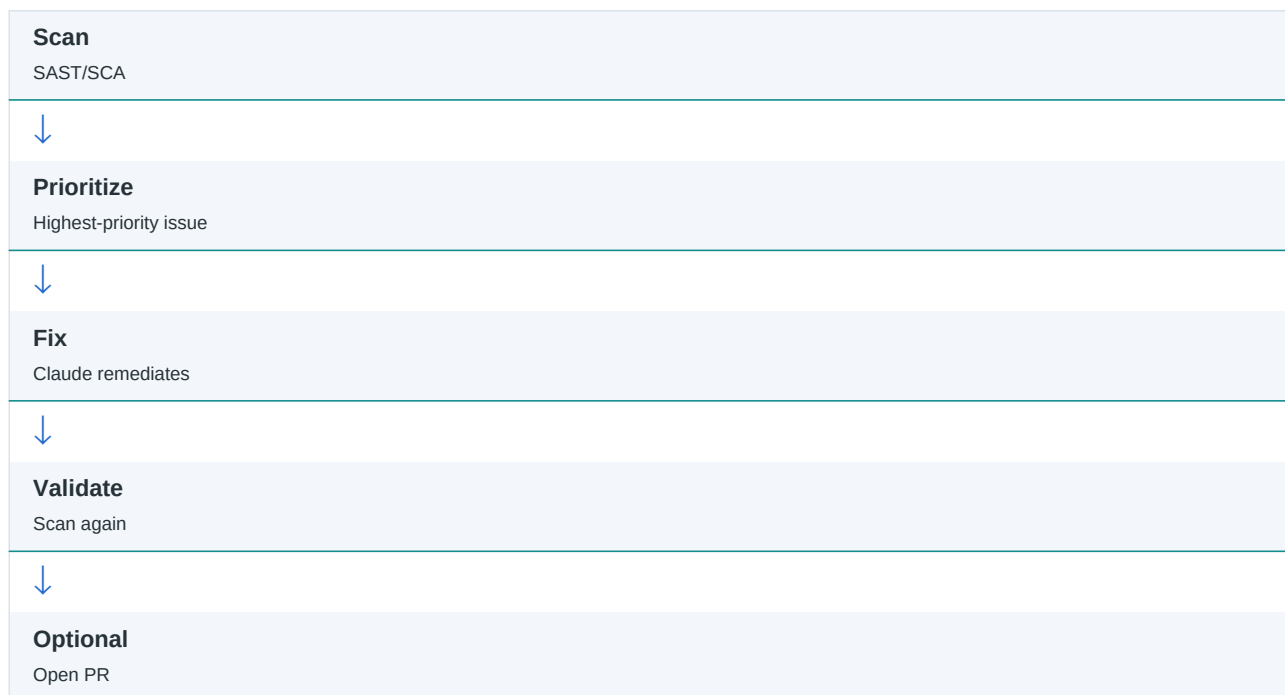
Developer asks Claude to create a login endpoint. The document illustrates an SQL query built by concatenating a user-controlled value, producing an SQL-injection risk.

```
"SELECT * FROM users WHERE name = " + username + ""
```

Responsibility split
Snyk supplies security analysis/findings; Claude supplies reasoning, code changes and remediation in the described workflow.

13. Other Capabilities Around MCP

/snyk-fix



Secure Dependency Health Check

The document says this evaluates packages when Claude is about to add a dependency, considering factors such as vulnerability history, maintenance health and popularity using Snyk package intelligence.

Tool profiles

- The source describes Lite, Full and experimental profiles.
- Profiles can limit the capabilities exposed to the AI agent.
- Exact availability depends on profile/version.

Error handling

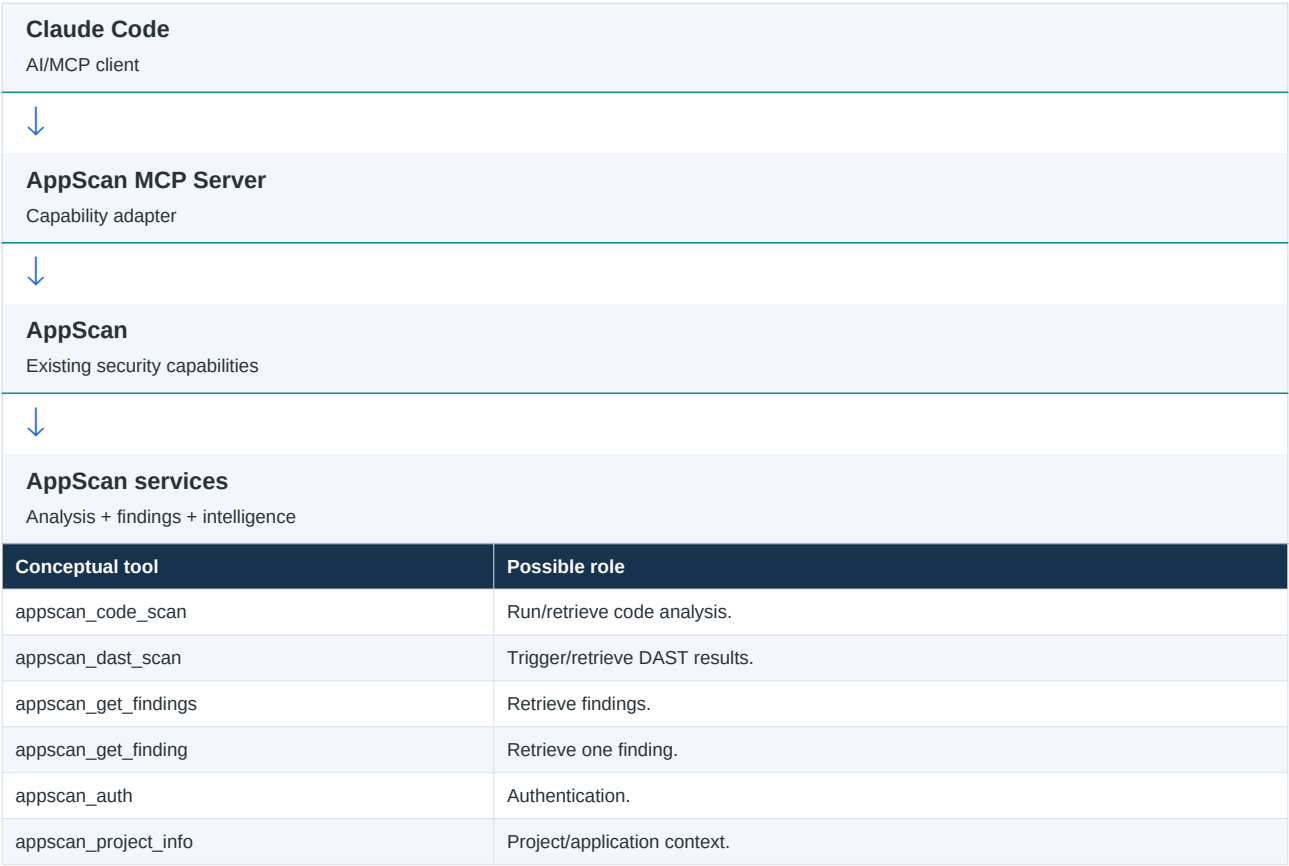
The MCP implementation contains handling for cases such as Snyk Code not being enabled. The broader lesson is to return meaningful, AI-consumable errors rather than raw command failures.

Big picture

The Snyk Claude experience is broader than MCP: capabilities + guidance + hooks/commands + security guardrails.

14. What Snyk Teaches Us About AppScan

The source's AppScan section is conceptual. It does not claim that the following tools already exist; it proposes the shape an integration could take.



- Reuse the existing security engine; do not build a Claude-specific engine.
- Prefer specialized tools.
- Make tool descriptions detailed.
- Return AI-friendly findings.
- Handle authentication explicitly.
- Protect local resources.
- Keep capability and workflow separate.

15. Final Mental Model + Meeting Cheat Sheet

The simplest way to explain the complete document



Snyk

Security analysis/intelligence



Hooks / directives / plugin

Workflow + enforcement

The one flow to remember

Scan → analyze → fix → re-scan

Claude changes code → security capability is invoked → Snyk produces a finding → the MCP adapter returns useful information → Claude reasons/remediates → the workflow can validate again.

If someone asks “What should AppScan copy from this?”

- Expose existing AppScan capabilities through a clean MCP server.
- Design focused tools rather than one generic scan tool.
- Write tool descriptions specifically for an LLM.
- Normalize security results before returning them.
- Provide authentication/project context capabilities.
- Define trust and local-resource boundaries.
- Add rules/hooks/workflow separately if the goal is continuous agentic security.

30-second explanation

“Claude Code is the MCP client. Snyk exposes security capabilities through an MCP server. Claude invokes those tools; the server checks prerequisites, runs the Snyk capability, processes the result and returns it. Snyk's surrounding plugin and hooks then connect those capabilities to the coding loop.”

This summary is based on the uploaded 51-page analysis. It intentionally compresses the source into the architecture, execution flow, controls, examples and AppScan lessons you need to understand the whole document.